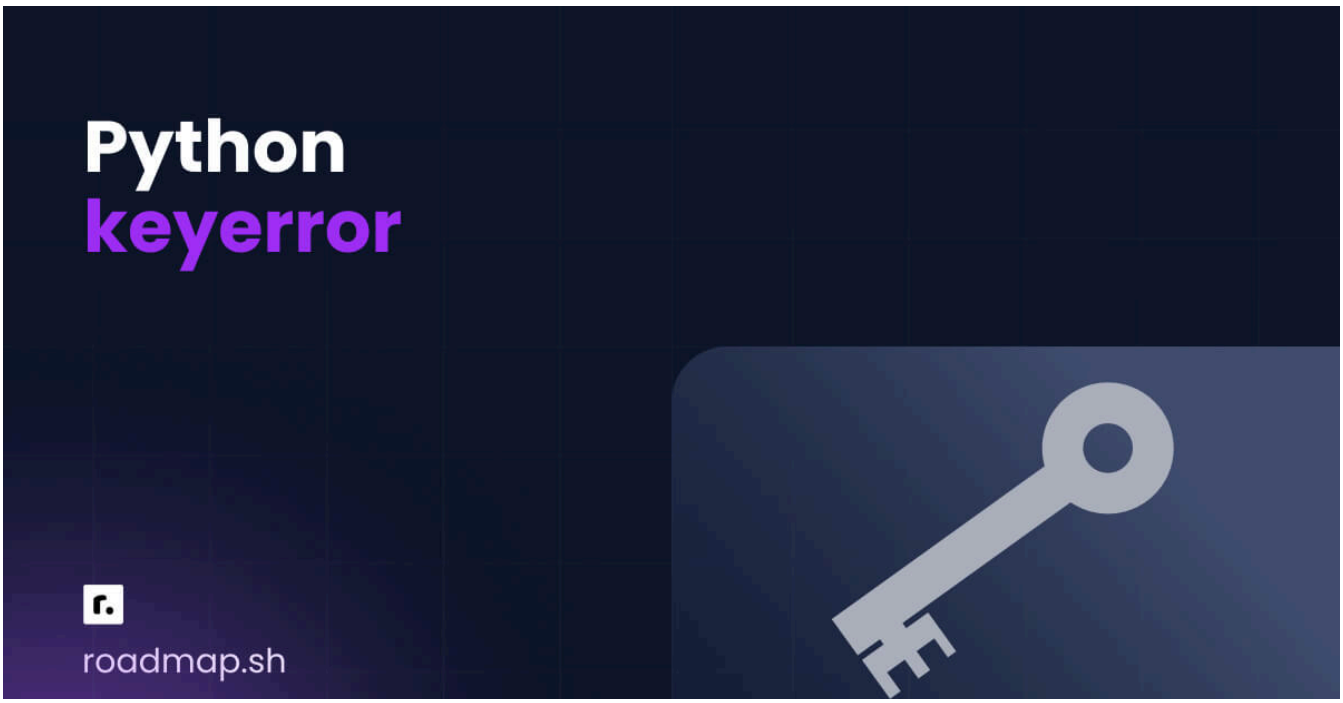
Исключения KeyError в Python: причины и способы устранения

Кимберли Фессел



Давайте посмотрим правде в глаза: ошибки в Python могут изрядно раздражать. Вы потратили время на то, чтобы отточить логику кода и исправить синтаксис, и вдруг столкнулись с ошибкой, которая останавливает работу программы. Одно из распространенных исключений — `KeyError`, которое возникает, когда вы пытаетесь получить доступ к несуществующему ключу словаря. Это расстраивает, но ошибки — не всегда плохо. Если вы потратите время на то, чтобы понять, что означают исключения в Python, почему они возникают и как их исправить, они станут ценными инструментами для отладки.

Содержание

В этой статье мы подробно рассмотрим исключение KeyError в Python. Мы покажем вам, как читать `KeyError` трассировку, и объясним наиболее распространенные ситуации, в которых возникает `KeyError` . Мы также пошагово разберем решения для `KeyError` обработки исключений. Для начала давайте разберемся, что такое `KeyError` и почему Python их генерирует.

Что такое исключение KeyError?

Ошибка Python `KeyError` возникает, когда вы пытаетесь получить доступ к ключу, которого нет в объекте сопоставления, например в **словаре**. Это исключение, которое сообщает, что ключ, на который вы ссылаетесь, отсутствует в сопоставлении. Вот как это может произойти:

python

```python
drink_prices = {"coffee": 3.50}

print(drink_prices["tea"])

# Expected result:
# KeyError: 'tea'
```

Словарь `drink_prices` не содержит ключа `'tea'` или связанного с ним значения, поэтому Python выдает ошибку `KeyError`.

Как прочитать трассировку KeyError

Когда Python выдает ошибку `KeyError`, он также генерирует трассировку, которая может помочь вам определить, где произошла ошибка и какой ключ вызвал проблему. Например, вот полная трассировка для нашего примера с `drink_prices`:

python

```python
drink_prices = {"coffee": 3.50}

print(drink_prices["tea"])

# Expected result (full traceback):
# Traceback (most recent call last):
#   File "<python-input-0>", line 3, in <module>
#     print(drink_prices["tea"])
#           ~~~~~~~~~~~~~^^^^^^^
# KeyError: 'tea'
```

Unlike syntax errors, which point to invalid Python code, `KeyError` tracebacks help you find where a lookup failed while the program was running. Here's a step-by-step approach to reading the traceback:

1. **Read the final line.** The last line of the traceback contains the exception type and additional details. Here, we have a `KeyError` and `'tea'` is the missing key.
2. **Find the line causing the error.** The traceback points to line 3, where Python attempted to access the `'tea'` key.
3. **Note the missing key.** Python can't find `'tea'`, so we either need to add it to our dictionary or not ask for it.

Once you've noted the line that triggered the error and which key is missing, the next step is determining why the key isn't present. Let's look at the most common causes of a `KeyError`.

Why Python KeyError Exceptions Occur: Causes

`KeyErrors` happen when you try to access a key that's absent from a mapping object, such as a dictionary or a Counter. Some non-mapping objects, including pandas DataFrames, may also raise a `KeyError` exception when a lookup fails. We'll focus on dictionaries throughout this section, but the root cause is similar in other objects. In most cases, the issue comes down to missing keys, type mismatches, unexpected whitespace, or case discrepancies.

Dictionary Key Doesn't Exist

The most common scenario for Python to raise a `KeyError` is when you reference a key that does not exist within a dictionary. Say you have a database of user login credentials. If a new user attempts to log in without signing up, the `KeyError` indicates that the requested key isn't in the dictionary:

python

```python
user_credentials = {
    "user001": "pAs$wurD",
    "user002": "password12345",
    "user003": "my_pass"
}

def login(username, password):
    if user_credentials[username] == password:
```

```
        return "Login successful"
    return "Login failed"

print(login("user004", "test"))

# Expected result:
# KeyError: 'user004'
```

The `user_credentials` dictionary does not contain the key `"user004"` and can't return that key's value. Therefore, Python raises a `KeyError` when it attempts to access it.

Type Mismatch

While many Python `KeyError` Some issues stem from straightforward missing keys, others are more subtle and therefore more difficult to debug. Consider the following code:

python

```
color_mapper = {
    "0": "black",
    "1": "blue",
    "2": "orange"
}

print(color_mapper[0])

# Expected result:
# KeyError: 0
```

This code attempts to find the color associated with zero, but fails because the keys in `color_mapper` are strings (`"0"` , `"1"` , `"2"`), not integers (`0`). Python requires an exact key match, including data type; otherwise, it raises a `KeyError` exception.

This type of issue often occurs when working with `pandas` DataFrames, where column labels may be strings while your code uses integers (or vice

versa). Before accessing data, it's important to verify the type and format of the keys or column names.

Whitespace or Case Sensitivity

Along with type mismatches, subtle formatting differences such as whitespace and letter casing can also lead to Python `KeyError` exceptions. Here's an illustrative example:

```python
student_scores = {
    " TOM ": 79,
    " TONY ": 92,
    " TERESA ": 84
}

print(student_scores["Tom"])

# Expected result:
# KeyError: 'Tom'
```

The keys of the dictionary include extra whitespace and uppercase letters (e.g. `" TOM "`), while the code attempts to access `"Tom"`. Both whitespace and letter case matter when looking up key values; ensure that you have both correct to avoid a `KeyError`.

> *AI Tutor: Show me five realistic examples of KeyErrors from type, whitespace, or case mismatches.*

Data from External Sources

KeyErrors often arise when working with data from external sources such as APIs, JSON responses, or some CSV files. These data structures aren't

always consistent, meaning keys may exist in one response but not in another. For instance:

```python
api_response = {
    "user": {
        "id": 1,
        "name": "Sheena"
    }
}

print(api_response["user"]["email"])

# Expected result:
# KeyError: 'email'
```

The `"email"` field wasn't included in the external data source, so attempting to get the key's value leads to a `KeyError`.

KeyError Exception Handling Strategies: Fixes

You've seen several situations that can cause a Python `KeyError`. Now we'll show you practical strategies to handle `KeyError` exceptions and prevent them from occurring in the first place.

The .get() Method with a Default

Instead of accessing a dictionary key directly, you can use the .get() method. Here's how it works:

```python
drink_prices = {"coffee": 3.50}

print(drink_prices.get("coffee"))

# Expected result:
```

```
# 3.5
```

The `.get()` method behaves like direct dictionary access (`drink_prices["coffee"]`) when the key exists. However, unlike direct access, it does not raise a `KeyError` exception when the key is missing. Instead, it returns **None** by default, or a value you specify.

For example, you can provide a fallback value if the key is not found:

python

```python
drink_prices = {"coffee": 3.50}

print(drink_prices.get("tea", 4.00))

# Expected result:
# 4.0
```

Because `"tea"` is not in the dictionary, Python returns the default value `4.00` instead of raising a `KeyError`.

Use `.get()` when:

- A missing key is expected or acceptable
- You want a safe value instead of an error
- You're working with incomplete data, say, from an API

Do not use `.get()` when a `KeyError` helps you catch problems earlier:

- A missing key means you have a bug in your code
- You must have the key for correct program behavior
- You immediately want to know about failures

Using `in` to Check for Keys

When your logic requires more control, you can include a conditional statement to check whether a key exists in a dictionary before proceeding. This allows you to handle missing keys using custom logic rather than returning a default value. For instance, you can warn the user that their beverage isn't in the `drink_prices` dictionary as well as falling back on a default value:

```python
drink_prices = {"coffee": 3.50}

drink = "tea"

if drink in drink_prices:
    price = drink_prices[drink]
else:
    print(f"Warning: {drink} not found in price list. Using default.")
    price = 4.00

print(price)

# Expected result:
# Warning: tea not found in price list. Using default.
# 4.0
```

We avoid the `KeyError` here by checking whether the key exists before attempting to access it. Since `"tea"` isn't in `drink_prices`, Python executes the `else` branch instead of directly finding its value in the dictionary.

Do use `in` to check for keys when:

- You need different behavior for a missing key, besides a simple default value

- You want to log, warn, or track missing keys
- Missing keys are meaningful for your program

Do not use `in` to check for keys when:

- A missing key means you have a bug in your code
- You immediately want to know about failures
- A straightforward `.get()` default value will do
- It introduces extra branching for no meaningful gain

try / except

Unlike `.get()` and `in`, which prevent a `KeyError` from happening in the first place, a `try/except` block allows the error to happen and then performs exception handling directly. The following example shows how that would work for `drink_prices`:

python

```python
drink_prices = {"coffee": 3.50}

try:
    price = drink_prices["tea"]
except KeyError:
    price = 4.00

print(price)

# Expected result:
# 4.0
```

Python attempts to access `"tea"`, which raises a `KeyError` exception. Instead of crashing, the program immediately transfers control to the

`except KeyError` block, where we assign a fallback value. We don't prevent the error, but rather, respond to it after it occurs.

As a best practice, it's recommended to explicitly specify the exception type in the `except` block (e.g. `except KeyError`) rather than using a bare `except`. This makes sure you only catch the error you expect.

Use `try/except` when:

- Missing keys are expected
- You need custom fallback logic or messaging
- You want your program to continue after a missing key
- You need to handle failure cases differently from regular keys

Do not use `try/except` when:

- A missing key means you have a bug in your code
- You immediately want to know about failures
- You are only using it to silently suppress errors
- `.get()` or `in` provides a cleaner solution

KeyError in Pandas

If you're using the pandas library for data handling, a `KeyError` typically indicates that a column does not exist. This Python snippet builds a small

pandas DataFrame and attempts to get the "age" column, which does not exist:

```python
import pandas as pd

df = pd.DataFrame([
    {"name": "Sonny", "zip": "32814"},
    {"name": "Neela", "zip": "60602"},
])

print(df["age"])

# Expected result:
# KeyError: 'age'
```

Although a pandas DataFrame isn't a dictionary, column selection using square brackets behaves like key-based lookup, hence the `KeyError` for missing columns. In practice, many pandas `KeyError` issues come from subtle mismatches in column labels, such as incorrect casing (*"Age"* versus *"age"*), unexpected whitespace (*"age "*), or type mismatches in column names.

When to Raise the Built-in Exception KeyError Deliberately

Like other built-in exception classes, you may choose to raise a `KeyError` in your own code to alert the user to invalid or incomplete input. Python developers often do this when a required key is missing from input data or if downstream logic cannot safely continue without it.

Here's a function that verifies four necessary configuration components and deliberately raises a `KeyError` if any are missing:

python

```python
def verify_config(config):
    required_keys = ["host", "port", "username", "password"]

    for key in required_keys:
        if key not in config:
            raise KeyError(f"Missing required config key: {key}")

    return config

config = {
    "host": "localhost",
    "port": 5432,
    "username": "admin"
}

verify_config(config)

# Expected result:
# KeyError: 'Missing required config key: password'
```

Intentionally raising a `KeyError` is the right move when missing data violates your expected structure. It lets you know about the issue immediately, and you can customize it with a clear, descriptive error message for debugging.

Wrapping Up

Instead of being frustrated by Python's `KeyError` exceptions, consider them a healthy part of the debugging process, ensuring your code functions properly without silent errors. They pop up most frequently for missing keys in a Python dictionary, but they can also appear for subtler issues like data type, whitespace, and case mismatch. Check the traceback as your first line of defence for `KeyError` exception handling, and try

other options like the `.get()` method, the `in` keyword, or a `try/except` block to handle them in your Python code.

Think you can spot `KeyError` issues and prevent them in your code? Test out your skills by interacting with the AI Tutor:

AI Tutor | Quiz me on KeyError exceptions in Python. >

Related Guides

Python Null (None): Guide to Missing Values and NoneType

Python Backend Development: Build Your First API

Python Backend Frameworks: How to Choose the Right One

Python Return Multiple Values: 4 Methods & Examples

Python Multiline Strings: The Complete Guide

Python not Operator: The Complete Guide to Logical Negation

Python Print New Line: Methods, Examples, and Best Practices

Python Exponent: 5 Methods for Exponentiation + Applications

Python Modulo Operator (%): Complete Guide with Examples

Python Division: Operators, Floor Division, and Examples

359K
GitHub Stars

Star us on GitHub
Help us reach #1

+2.8M
Registered Users

Register yourself
Commit to your growth

49K
Discord Members

Join on Discord
Join the community

Roadmaps Guides FAQs YouTube

roadmap.sh by @nilbuild

Community created roadmaps, best practices, projects, articles, resources and journeys to help you choose your path and grow in your career.

THENEWSTACK

The top DevOps resource for Kubernetes, cloud-native computing, and large-scale development and deployment.

DevOps · Kubernetes · Cloud-Native

Cookie Settings